

교과목 4 : AI 활용 애플리케이션 개발

단원 2 : Django Framework

DAY4. Django 모델 설계와 내부 동작 구조

목 차

1. Django 모델이란?		3
2. 좋은 모델을 설계하는 순서		5
3. 모델 클래스가 만들어지는 내부 과정		9
4. Django가 제공하는 주요 Field 종류		14
5. Field 인수와 선택 기준		23
6. 무결성 제약조건 적용		32
7. 동시성과 트랜잭션		38

1. Django 모델이란?

Django 의 모델(Model)은 애플리케이션에서 다루는 데이터의 구조, 관계, 제약조건, 일부 행위를 Python 클래스로 표현한 것입니다.

일반적으로 다음과 같이 대응됩니다.

Django 구성 요소	관계형 데이터베이스
모델 클래스	테이블
모델 인스턴스	행(row)
모델 필드	열(column)
ForeignKey	외래 키
Meta.constraints	제약조건
Meta.indexes	인덱스
Manager, QuerySet	질의 생성 인터페이스

각 모델은 `django.db.models.Model` 을 상속하며, 클래스 속성으로 선언한 `Field` 가 데이터베이스 열이 됩니다. 기본 키를 직접 선언하지 않으면 Django 가 자동으로 기본 키를 추가합니다.

[Django 모델 공식 문서](#)

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    student_number = models.CharField(max_length=20, unique=True)
    admitted_at = models.DateField()
```

대략 다음과 같은 테이블을 표현합니다.

```
CREATE TABLE student (
    id BIGINT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    student_number VARCHAR(20) NOT NULL UNIQUE,
    admitted_at DATE NOT NULL
);
```

실제 SQL 자료형과 문법은 PostgreSQL, MySQL, SQLite 등 데이터베이스 백엔드에 따라 달라집니다.

2. 좋은 모델을 설계하는 순서

1. 업무 개념을 엔터티로 분리한다

먼저 프로그램이 관리할 대상을 찾습니다.

대학교 수강 관리 시스템이라면 다음과 같은 엔터티가 있습니다.

- 학생
- 교수
- 학과
- 교과목
- 강좌
- 수강 신청

여기서 “교과목”과 “강좌”는 구분하는 편이 좋습니다.

- 교과목: 데이터베이스개론이라는 과목 자체
- 강좌: 2026 년 1 학기에 김교수가 개설한 데이터베이스개론 01 분반

업무상 별개의 생명주기와 속성을 가진 대상을 하나의 테이블에 모두 넣으면 중복과 갱신 이상이 발생합니다.

2. 속성의 의미에 맞는 자료형을 정한다

예를 들어 학점은 정수이므로 IntegerField, 금액은 정확한 십진 연산이 필요하므로 DecimalField, 설명문은 길이가 가변적이므로 TextField 가 적절합니다.

```
credits = models.PositiveSmallIntegerField()
tuition = models.DecimalField(max_digits=10, decimal_places=0)
description = models.TextField(blank=True)
```

잘못된 예는 모든 값을 문자열로 저장하는 것입니다.

권장하지 않음

```
credits = models.CharField(max_length=10)
tuition = models.CharField(max_length=20)
```

이렇게 하면 숫자 비교, 합계, 정렬, 범위 검사에서 문제가 생깁니다.

3. 관계의 카디널리티를 결정한다

다대일: ForeignKey

여러 학생이 한 학과에 소속될 수 있습니다.

```
class Department(models.Model):
    name = models.CharField(max_length=100, unique=True)

class Student(models.Model):
    department = models.ForeignKey(
        Department,
        on_delete=models.PROTECT,
        related_name="students",
    )
```

- student.department: 학생의 학과
- department.students.all(): 해당 학과의 모든 학생

일대일: OneToOneField

한 사용자에게 프로필이 최대 하나만 존재하는 경우입니다.

```
class Profile(models.Model):
    user = models.OneToOneField(
        "auth.User",
        on_delete=models.CASCADE,
        related_name="profile",
    )
```

데이터베이스 관점에서는 고유 제약조건이 붙은 외래 키와 유사합니다.

다대다: ManyToManyField

한 학생은 여러 강좌를 수강하고, 한 강좌에도 여러 학생이 참여합니다.

단순 연결만 필요하다면 다음처럼 선언합니다.

```
class CourseSection(models.Model):
    students = models.ManyToManyField(
        Student,
        related_name="sections",
    )
```

하지만 수강일, 성적, 상태 같은 “관계 자체의 속성”이 있으면 중간 모델을 명시해야 합니다.

```
class Enrollment(models.Model):
    student = models.ForeignKey(
        Student,
        on_delete=models.CASCADE,
        related_name="enrollments",
    )
    section = models.ForeignKey(
        "CourseSection",
        on_delete=models.CASCADE,
        related_name="enrollments",
    )
    enrolled_at = models.DateTimeField(auto_now_add=True)
    grade = models.CharField(max_length=2, blank=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["student", "section"],
                name="uq_enrollment_student_section",
            )
        ]

class CourseSection(models.Model):
    students = models.ManyToManyField(
```

```
Student,  
    through=Enrollment,  
    related_name="sections",  
)
```

ManyToManyField 자체는 모델 테이블의 일반 열이 되지 않습니다. Django 가 자동 중간 테이블을 생성하거나, through 로 지정한 모델이 연결 테이블 역할을 합니다.

3. 모델 클래스가 만들어지는 내부 과정

1. Python 클래스 선언 단계

Django 모델 클래스는 일반 Python 클래스처럼 보이지만 내부적으로 메타클래스인 `ModelBase` 의 처리를 받습니다.

```
class Student(models.Model):  
    name = models.CharField(max_length=100)
```

클래스가 정의될 때 대략 다음 과정이 일어납니다.

1. `CharField` 객체가 클래스 속성으로 전달됩니다.
2. `ModelBase` 가 필드들을 수집합니다.
3. 필드 이름, 모델 참조, 열 이름 등의 정보가 설정됩니다.
4. 필드 정보가 `Student._meta` 에 등록됩니다.
5. 기본 `Manager` 인 `objects` 가 연결됩니다.
6. 관계 접근을 위한 `descriptor` 가 설치됩니다.
7. 앱 레지스트리에 모델이 등록됩니다.

`_meta` 를 사용하면 모델 구조를 실행 중에 조사할 수 있습니다.

```
Student._meta.db_table  
Student._meta.pk  
Student._meta.get_field("name")  
Student._meta.get_fields()
```

`_meta` 는 관리자 페이지, `ModelForm`, 직렬화, 마이그레이션 감지 등 Django 의 여러 기능에서 사용됩니다.

2. 모델 속성과 `descriptor`

`student.department` 를 읽으면 단순히 정수형 외래 키 값이 반환되는 것이 아니라 관련 `Department` 객체가 반환됩니다.

```
student.department      # Department 객체  
student.department_id   # 실제 FK 값
```

Django 가 모델 클래스에 관계 descriptor 를 설치하기 때문입니다. 관련 객체가 아직 로드되지 않았다면 쿼리를 실행하고, 이미 캐시되어 있으면 캐시된 객체를 반환할 수 있습니다. 따라서 외래 키 값만 필요할 때는 다음 표현이 효율적입니다.

```
student.department_id
```

student.department.id 는 경우에 따라 학과 객체를 가져오는 추가 쿼리를 발생시킬 수 있습니다.

3. 마이그레이션의 내부 구조

모델 클래스를 수정했다고 데이터베이스가 즉시 변경되지는 않습니다.

```
python manage.py makemigrations
python manage.py migrate
```

3.1 makemigrations

makemigrations 는 현재 모델 상태와 기존 마이그레이션이 표현하는 과거 상태를 비교하고, 차이를 마이그레이션 파일로 기록합니다.

```
class Migration(migrations.Migration):
    dependencies = [
        ("university", "0001_initial"),
    ]

    operations = [
        migrations.AddField(
            model_name="student",
            name="email",
            field=models.EmailField(max_length=254, null=True),
        ),
    ]
```

중요한 점은 Django 가 운영 데이터베이스를 직접 비교해 마이그레이션을 만드는 것이 아니라, 일반적으로 "모델 코드"와 "마이그레이션이 재구성한 모델 상태"를 비교한다는 것입니다.

3.2 migrate

migrate 는 마이그레이션 의존성 그래프를 계산하고, 적용되지 않은 작업을 데이터베이스별 SQL 로 변환해 실행합니다.

마이그레이션은 데이터베이스 스키마를 위한 버전 관리 기록과 비슷합니다. 따라서 모델 코드와 마이그레이션 파일을 함께 형상 관리해야 합니다. [Django 마이그레이션 공식 문서](#)

유용한 명령은 다음과 같습니다.

```
python manage.py showmigrations
python manage.py sqlmigrate university 0002
python manage.py migrate university 0001
```

데이터 마이그레이션에서는 현재 모델을 직접 import 하지 말고 역사적 모델을 사용해야 합니다.

```
def forwards(apps, schema_editor):
    Student = apps.get_model("university", "Student")
    Student.objects.filter(status="N").update(status="NEW")
```

4. ORM 질의의 내부 동작

4.1 Manager 와 QuerySet

```
students = Student.objects.filter(
    department__name="컴퓨터공학과"
).order_by("name")
```

- objects: 기본 Manager
- filter(): 조건이 추가된 새 QuerySet 반환
- department__name: 관계를 따라가며 JOIN 조건 생성
- order_by(): 정렬 조건 추가

대부분의 QuerySet 은 지연 평가됩니다. 위 코드만으로는 보통 SQL 이 즉시 실행되지 않습니다. 다음과 같이 결과가 필요한 순간에 실행됩니다.

```
list(students)
for student in students:
    print(student.name)
```

```
students.exists()
students.count()
students.first()
```

일반적인 흐름은 다음과 같습니다.

```
QuerySet 구성
→ 질의 표현 객체 생성
→ 데이터베이스 백엔드용 SQL 컴파일
→ SQL 과 바인딩 파라미터 실행
→ 결과 행을 모델 인스턴스로 변환
```

조건 값은 SQL 문자열에 직접 연결하지 않고 파라미터로 바인딩됩니다.

```
Student.objects.filter(name=user_input)
```

따라서 일반적인 ORM 조회는 SQL 삽입 공격을 막는 데 도움이 됩니다. 하지만 `raw()`, `RawSQL`, `extra()` 또는 직접 작성한 SQL에서는 별도 주의가 필요합니다.

4.2 N+1 문제

```
students = Student.objects.all()

for student in students:
    print(student.department.name)
```

학생 조회 1 회 후 각 학생의 학과를 개별 조회하면 N+1 문제가 생길 수 있습니다. 외래 키와 일대일 관계에는 `select_related()`를 사용합니다.

```
students = Student.objects.select_related("department")
```

다대다 또는 역방향 관계에는 `prefetch_related()`를 사용합니다.

```
departments = Department.objects.prefetch_related("students")
```

- `select_related()`: SQL JOIN 을 이용해 한 질의로 가져옴
- `prefetch_related()`: 별도 질의를 실행한 뒤 Python 에서 연결

4.3 저장과 삭제

```
student = Student(name="홍길동")
student.save()
```

새 객체인지 기존 객체인지와 기본 키 상태 등에 따라 INSERT 또는 UPDATE 가 실행됩니다. 주의할 점은 save()가 자동으로 full_clean()을 호출하지 않는다는 것입니다.

```
student.full_clean()
student.save()
```

ModelForm 은 유효성 검사를 수행하지만, 코드에서 직접 모델 객체를 생성하거나 save()할 때는 필요한 검증을 명시적으로 수행해야 합니다. 최종적인 데이터 무결성은 데이터베이스 제약조건으로도 보장해야 합니다.

또한 다음 연산은 모델의 개별 save()를 호출하지 않습니다.

```
Student.objects.filter(active=False).update(status="INACTIVE")
Student.objects.bulk_create(objects)
```

따라서 save() 재정의만으로 중요한 업무 규칙을 보장하면 우회될 수 있습니다.

4. Django가 제공하는 주요 Field 종류

전체 필드 API 는 [Django 필드 공식 레퍼런스](#)에서 확인할 수 있습니다.

1. 기본 키 필드

AutoField

자동 증가하는 32 비트 정수형 기본 키입니다. 신규 프로젝트에서는 보통 `BigAutoField` 가 기본값입니다.

BigAutoField

자동 증가하는 64 비트 정수형 기본 키입니다. 행이 많이 생성될 가능성이 있는 일반적인 서비스에 적합합니다.

```
id = models.BigAutoField(primary_key=True)
```

대부분은 직접 선언할 필요가 없습니다.

SmallAutoField

작은 범위의 자동 증가 정수입니다. 생성 가능한 행 수가 명확히 작을 때만 고려합니다.

CompositePrimaryKey

Django 5.2 부터 지원된 복합 기본 키입니다.

```
class Release(models.Model):
    pk = models.CompositePrimaryKey("version", "name")
    version = models.IntegerField()
    name = models.CharField(max_length=20)
```

레거시 데이터베이스나 관계 자체가 자연스럽게 복합 식별자를 갖는 경우 유용합니다. 하지만 외부 라이브러리, 관리자 기능, 관계 처리의 지원 범위를 확인해야 하므로 일반 애플리케이션에서는 단일 대리 키와 `UniqueConstraint` 가 더 단순할 수 있습니다.

2. 문자열 필드

CharField

길이가 제한된 문자열입니다.

```
name = models.CharField(max_length=100)
```

적합한 용도:

- 이름
- 제목
- 코드
- 짧은 주소
- 상태 값

max_length 는 데이터베이스 열 정의와 Django 유효성 검사에 사용됩니다.

TextField

길이가 긴 문자열입니다.

```
description = models.TextField(blank=True)
```

적합한 용도:

- 게시글 본문
- 긴 설명
- 메모

검색과 정렬이 빈번한 짧은 값에는 CharField 가 의미를 더 잘 표현합니다.

SlugField

URL 의 경로 조각에 적합한 문자열입니다.

```
slug = models.SlugField(max_length=150, unique=True)
```

예: django-model-guide

보통 영문자, 숫자, 밑줄, 하이픈을 검증하며 인덱스가 기본 설정됩니다.

EmailField

이메일 형태를 검증하는 CharField 입니다.

```
email = models.EmailField(unique=True)
```

형식 검사는 하지만 해당 이메일 주소의 실제 존재 여부를 보증하지는 않습니다.

URLField

URL 형식을 검증합니다.

```
homepage = models.URLField(blank=True)
```

GenericIPAddressField

IPv4 또는 IPv6 주소를 저장하고 검증합니다.

```
ip_address = models.GenericIPAddressField(  
    protocol="both",  
    unpack_ipv4=True,  
    null=True,  
    blank=True,  
)
```

3. 숫자 필드

IntegerField

일반적인 정수입니다.

```
capacity = models.IntegerField()
```

SmallIntegerField, BigIntegerField

저장 범위에 따라 선택합니다.

PositiveIntegerField, PositiveSmallIntegerField, PositiveBigIntegerField

음수가 허용되지 않는 정수에 적합합니다.


```
credits = models.PositiveSmallIntegerField()
```

그러나 양수 필드라도 "1 이상 6 이하"와 같은 업무 범위는 별도 validator 나 CheckConstraint 로 표현해야 합니다.

FloatField

부동소수점 수입입니다.

```
measurement = models.FloatField()
```

과학적 측정처럼 근삿값이 허용되는 경우에 사용합니다. 이진 부동소수점 오차가 있으므로 금액에는 부적합합니다.

DecimalField

정확한 고정소수점 수입입니다.

```
price = models.DecimalField(  
    max_digits=12,  
    decimal_places=2,  
)
```

- max_digits=12: 전체 최대 12 자리
- decimal_places=2: 소수점 이하 2 자리
- 정수 부분은 최대 10 자리

금액, 세율, 회계 데이터에는 일반적으로 FloatField 보다 DecimalField 가 적절합니다.

4. 논리 필드

BooleanField

참과 거짓을 저장합니다.

```
is_active = models.BooleanField(default=True)
```

null=True 를 추가하면 True, False, NULL 의 세 상태를 표현할 수 있습니다. 그러나 "미정"이라는 세 번째 상태가 실제 업무 의미를 가질 때만 사용해야 합니다.

5. 날짜와 시간 필드

DateField

날짜만 저장합니다.

```
birth_date = models.DateField()
```

TimeField

시간만 저장합니다.

```
starts_at = models.TimeField()
```

DateTimeField

날짜와 시간을 함께 저장합니다.

```
published_at = models.DateTimeField(null=True, blank=True)
```

USE_TZ=True 인 프로젝트에서는 시간대 인식 값을 사용하고 `django.utils.timezone.now` 를 기본값으로 쓰는 것이 안전합니다.

```
from django.utils import timezone

created_at = models.DateTimeField(default=timezone.now)
```

auto_now_add 와 auto_now

```
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
```

- `auto_now_add=True`: 처음 생성될 때 설정
- `auto_now=True`: `save()`될 때마다 갱신

이 옵션들은 필드를 `editable=False` 로 만들며, `QuerySet.update()` 같은 연산에서는 자동 갱신되지 않습니다. 시간을 직접 제어해야 하거나 테스트 유연성이 중요하면 `default=timezone.now` 를 고려할 수 있습니다.

DurationField

시간 간격을 나타냅니다.

```
duration = models.DurationField()
```

Python에서는 `datetime.timedelta`로 다룹니다. 데이터베이스마다 표현과 연산 지원에 차이가 있을 수 있습니다.

6. 파일 필드

FileField

파일 자체보다 일반적으로 저장소의 파일 경로 또는 식별자를 데이터베이스에 저장합니다.

```
document = models.FileField(upload_to="documents/%Y/%m/")
```

ImageField

이미지 크기 검증 등의 기능이 추가된 FileField입니다.

```
photo = models.ImageField(
    upload_to="profiles/",
    blank=True,
)
```

이미지 처리를 위해 Pillow가 필요합니다.

파일 삭제 시 실제 저장소의 파일이 자동으로 삭제될 것이라고 가정하면 안 됩니다. 보존 정책, 교체, 삭제 처리를 별도로 설계해야 합니다.

7. 구조화 및 특수 데이터 필드

JSONField

JSON 객체를 저장합니다.

```
metadata = models.JSONField(default=dict, blank=True)
```

좋은 사용 사례:

- 구조가 유동적인 부가 정보
- 외부 API 의 원본 응답
- 자주 변경되는 선택적 설정

부적절한 사용 사례:

- 외래 키가 필요한 핵심 업무 데이터
- 자주 JOIN 하거나 강한 타입 및 제약이 필요한 값
- 모든 속성을 JSON 하나에 몰아넣는 설계

가변 객체를 기본값으로 직접 넣으면 안 됩니다.

잘못된 방식

```
metadata = models.JSONField(default={})
```

올바른 방식

```
metadata = models.JSONField(default=dict)
```

BinaryField

bytes 와 같은 원시 이진 데이터를 저장합니다. 일반 파일 저장 용도로는 권장되지 않습니다.

UUIDField

UUID 를 저장합니다.

```
import uuid

public_id = models.UUIDField(
    default=uuid.uuid4,
    unique=True,
    editable=False,
)
```

외부에 노출되는 식별자를 예측하기 어렵게 만들거나 분산 환경에서 애플리케이션이 식별자를 생성해야 할 때 유용합니다. 하지만 UUID 는 정수형 키보다 크며 인덱스 지역성과 가독성 측면에서 비용이 있을 수 있습니다.

GeneratedField

데이터베이스가 다른 열을 이용해 계산하는 생성 열을 표현합니다.

```
from django.db.models import F

total = models.GeneratedField(
    expression=F("price") * F("quantity"),
    output_field=models.DecimalField(
        max_digits=14,
        decimal_places=2,
    ),
    db_persist=True,
)
```

데이터베이스별 지원 여부와 표현식 제한을 확인해야 합니다.

8. 관계 필드

ForeignKey

다대일 관계입니다.

```
department = models.ForeignKey(
    Department,
    on_delete=models.PROTECT,
)
```

OneToOneField

일대일 관계입니다.

```
profile = models.OneToOneField(  
    Profile,  
    on_delete=models.CASCADE,  
)
```

ManyToManyField

다대다 관계입니다.

```
tags = models.ManyToManyField(Tag, blank=True)
```

ManyToManyField에서는 `null=True`가 의미 없습니다. 관계가 없다는 것은 중간 테이블에 행이 없다는 뜻이기 때문입니다.

5. Field 인수와 선택 기준

1. null

```
nickname = models.CharField(max_length=50, null=True)
```

null=True 는 데이터베이스가 SQL NULL 을 허용한다는 의미입니다.

문자열 필드에서는 일반적으로 다음처럼 사용합니다.

```
nickname = models.CharField(max_length=50, blank=True)
```

문자열의 “값 없음”을 빈 문자열 ""로 통일하기 위해서입니다. null=True 까지 사용하면 NULL 과 ""라는 두 종류의 빈 값이 생깁니다.

예외적으로 unique=True, blank=True 인 문자열에 여러 빈 값을 허용하려면 데이터베이스의 NULL 처리 특성을 고려하여 null=True 가 필요할 수 있습니다. [필드 옵션 공식 문서](#)

2. blank

```
bio = models.TextField(blank=True)
```

blank=True 는 폼과 모델 유효성 검사에서 빈 입력을 허용한다는 뜻입니다.

- null: 데이터베이스 저장 규칙
- blank: Django 유효성 검사 규칙

예를 들어 선택적인 날짜 필드는 보통 둘 다 필요합니다.

```
graduated_at = models.DateField(  
    null=True,  
    blank=True,  
)
```

3. default

```
status = models.CharField(max_length=20, default="PENDING")
```

고정값이나 호출 가능한 객체를 줄 수 있습니다.

```
created_at = models.DateTimeField(default=timezone.now)
public_id = models.UUIDField(default=uuid.uuid4)
metadata = models.JSONField(default=dict)
```

함수 호출 결과를 쓰면 안 됩니다.

잘못된 예: 서버가 시작될 때 한 번 계산될 수 있음

```
created_at = models.DateTimeField(default=timezone.now())
```

올바른 예: 객체를 만들 때마다 호출

```
created_at = models.DateTimeField(default=timezone.now)
```

람다를 마이그레이션에 직렬화할 수 없으므로 사용하지 않는 것이 원칙입니다.

4. db_default

데이터베이스 수준 기본값입니다.

```
from django.db.models.functions import Now

created_at = models.DateTimeField(db_default=Now())
```

- default: Python/Django 가 값을 결정
- db_default: 데이터베이스가 값을 결정

두 값을 모두 지정하면 ORM 을 통한 객체 생성에서는 default 가 우선하지만, 데이터베이스 기본값도 스키마에 남습니다.

5. unique

```
student_number = models.CharField(
    max_length=20,
    unique=True,
)
```

데이터베이스에 UNIQUE 제약조건이 생성됩니다. 단일 필드의 무조건적인 고유성에 적합합니다. unique=True 는 인덱스를 수반하므로 보통 db_index=True 를 함께 지정할 필요가 없습니다.

6. primary_key

```
code = models.CharField(  
    max_length=20,  
    primary_key=True,  
)
```

기본 키는 자동으로 `null=False`, `unique=True` 가 됩니다.

자연 키를 기본 키로 삼으면 그 값의 변경이 모든 관계에 영향을 미칠 수 있습니다. 학번, 이메일, 주민등록번호처럼 업무상 변경 가능성이 있는 값은 별도의 고유 필드로 두고 자동 증가 기본 키를 사용하는 편이 대체로 안전합니다.

7. choices

고정된 선택지를 제한합니다.

```
class Student(models.Model):  
    class Status(models.TextChoices):  
        ACTIVE = "ACTIVE", "재학"  
        LEAVE = "LEAVE", "휴학"  
        GRADUATED = "GRADUATED", "졸업"  
  
    status = models.CharField(  
        max_length=10,  
        choices=Status,  
        default=Status.ACTIVE,  
    )  
    student.get_status_display()
```

`choices` 는 Django 의 모델 검증과 폼 선택 목록에 사용됩니다. 자주 바뀌거나 관리자가 추가해야 하는 값은 `choices` 보다 별도 모델과 `ForeignKey` 가 적절합니다.

또한 `choices` 만으로 모든 데이터베이스 입력 경로를 강제한다고 가정해서는 안 됩니다. DB 수준 보장이 필요하면 `CheckConstraint` 도 고려합니다.

8. validators

```
from django.core.validators import MinValueValidator, MaxValueValidator

credits = models.IntegerField(
    validators=[
        MinValueValidator(1),
        MaxValueValidator(6),
    ]
)
```

validator 는 Django 유효성 검사 계층에서 작동합니다. SQL 에서 직접 삽입하거나 save() 전에 검증을 호출하지 않으면 우회될 수 있으므로 핵심 무결성 규칙은 DB 제약조건도 함께 사용합니다.

9. db_index

```
student_number = models.CharField(
    max_length=20,
    db_index=True,
)
```

단일 필드 인덱스를 만듭니다. Django 6.0 문서는 기능이 더 풍부한 Meta.indexes 를 우선 고려하도록 안내합니다.

인덱스는 조회를 빠르게 할 수 있지만 다음 비용이 있습니다.

- 저장 공간 증가
- INSERT, UPDATE, DELETE 비용 증가
- 잘못된 열이나 순서에 만든 인덱스는 사용되지 않을 수 있음

따라서 “검색할 것 같으니 전부 인덱스”는 좋은 설계가 아닙니다.

10. 기타 공통 인수

```
name = models.CharField(
    "학생 이름",                                # verbose_name
    max_length=100,
    help_text="공식 학적부의 이름",
    db_column="student_name",
    db_comment="학생의 공식 이름",
    editable=True,
)
```

- verbose_name: 관리자·폼에 표시할 이름
- help_text: 입력 도움말
- db_column: 실제 데이터베이스 열 이름
- db_comment: DB 열 설명
- editable=False: 관리자와 ModelForm 에서 편집하지 않음
- error_messages: 기본 검증 오류 메시지 변경
- serialize: Django 직렬화 대상 여부
- db_tablespace: 인덱스 테이블스페이스

11. 관계 필드의 핵심 인수

11.1 on_delete

참조 대상이 삭제될 때의 동작을 결정하는 필수 인수입니다.

CASCADE

부모가 삭제되면 자식도 삭제됩니다.

```
post = models.ForeignKey(
    Post,
    on_delete=models.CASCADE,
)
```

게시글이 삭제될 때 그 게시글의 댓글도 삭제해도 되는 경우에 적합합니다.

PROTECT

참조하는 객체가 존재하면 삭제를 금지하고 ProtectedError 를 발생시킵니다.

```
department = models.ForeignKey(  
    Department,  
    on_delete=models.PROTECT,  
)
```

학생이 남아 있는 학과를 삭제하면 안 되는 경우에 적합합니다.

RESTRICT

제한 삭제를 수행하며 RestrictedError 를 발생시킵니다. 같은 삭제 작업에서 함께 연쇄 삭제되는 관계를 고려한다는 점에서 PROTECT 와 세부 동작이 다릅니다.

SET_NULL

부모가 삭제되면 외래 키를 NULL 로 변경합니다.

```
advisor = models.ForeignKey(  
    Professor,  
    on_delete=models.SET_NULL,  
    null=True,  
    blank=True,  
)
```

반드시 null=True 여야 합니다.

SET_DEFAULT

부모가 삭제되면 기본값으로 변경합니다.

```
category = models.ForeignKey(  
    Category,  
    on_delete=models.SET_DEFAULT,  
    default=1,  
)
```

기본 대상이 실제로 존재하는지 관리해야 합니다.

SET(...)

지정한 값 또는 callable 의 반환값으로 변경합니다.

```
def get_sentinel_user():
    return User.objects.get_or_create(username="deleted")[0]

author = models.ForeignKey(
    User,
    on_delete=models.SET(get_sentinel_user),
)
```

DO_NOTHING

Django 는 아무 작업도 하지 않습니다. 데이터베이스 외래 키 제약조건에 의해 오류가 발생할 수 있습니다. 데이터베이스 동작을 정확히 이해하고 별도로 관리하는 경우가 아니면 신중해야 합니다.

11.2 related_name

역방향 관계 이름을 정합니다.

```
department = models.ForeignKey(
    Department,
    on_delete=models.PROTECT,
    related_name="students",
)

department.students.all()
```

지정하지 않으면 기본적으로 student_set 같은 이름이 생성됩니다.

역방향 관계가 필요 없다면 다음처럼 차단할 수 있습니다.

```
related_name="+"
```

11.3 related_query_name

역방향 필터 경로의 이름을 별도로 정합니다.

```
department = models.ForeignKey(
    Department,
    on_delete=models.PROTECT,
    related_name="students",
    related_query_name="student",
)
Department.objects.filter(student__name="홍길동")
```

11.4 to_field

대상 모델의 기본 키가 아닌 고유 필드를 참조합니다.

```
department = models.ForeignKey(
    Department,
    to_field="code",
    on_delete=models.PROTECT,
)
```

대상 필드에는 일반적으로 고유성이 보장되어야 합니다. 변경 가능한 업무 코드를 관계 키로 사용하면 갱신 비용이 커질 수 있습니다.

11.5 db_constraint

```
department = models.ForeignKey(
    Department,
    on_delete=models.DO_NOTHING,
    db_constraint=False,
)
```

False 이면 데이터베이스 외래 키 제약조건을 만들지 않습니다. 레거시 시스템 등 특별한 이유가 아니면 기본값 True 를 유지해야 합니다. ORM 관계가 존재한다고 해서 DB 무결성이 자동으로 보장되는 것은 아니기 때문입니다.

11.6 자기 자신 또는 아직 선언되지 않은 모델 참조

```
class Category(models.Model):
    parent = models.ForeignKey(
        "self",
        on_delete=models.CASCADE,
        null=True,
        blank=True,
        related_name="children",
    )
```

다른 앱의 모델도 문자열로 참조할 수 있습니다.

```
owner = models.ForeignKey(
    "accounts.User",
    on_delete=models.CASCADE,
)
```

사용자 모델은 보통 직접 "auth.User"를 참조하기보다 다음을 사용합니다.

```
from django.conf import settings

owner = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.CASCADE,
)
```

6. 무결성 제약조건 적용

무결성은 데이터가 업무 규칙을 계속 만족하도록 보장하는 성질입니다.

1. 개체 무결성

각 행을 고유하게 식별해야 합니다. 기본 키가 이를 담당합니다.

```
id = models.BigAutoField(primary_key=True)
```

2. 참조 무결성

외래 키 값은 실제 부모 행을 가리켜야 합니다.

```
department = models.ForeignKey(
    Department,
    on_delete=models.PROTECT,
)
```

3. 도메인 무결성

값이 허용된 자료형과 범위에 속해야 합니다.

```
credits = models.PositiveSmallIntegerField()
```

보다 구체적인 범위는 `CheckConstraint` 로 설정할 수 있습니다.

4. UniqueConstraint

둘 이상의 필드를 조합해 고유성을 보장합니다.

```
class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    section = models.ForeignKey(CourseSection, on_delete=models.CASCADE)

    class Meta:
        constraints = [
```



```

        models.UniqueConstraint(
            fields=["student", "section"],
            name="uq_enrollment_student_section",
        )
    ]

```

동일 학생이 동일 강좌에 두 번 등록되는 것을 데이터베이스가 막습니다.

조건부 고유 제약조건

```

from django.db.models import Q

class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    section = models.ForeignKey(CourseSection, on_delete=models.CASCADE)
    is_cancelled = models.BooleanField(default=False)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["student", "section"],
                condition=Q(is_cancelled=False),
                name="uq_active_enrollment",
            )
        ]

```

취소되지 않은 수강 신청만 고유하도록 만들 수 있습니다. 조건부 제약 지원은 데이터베이스 백엔드에 따라 차이가 있습니다.

표현식 기반 고유성

```

from django.db.models.functions import Lower

class Department(models.Model):
    name = models.CharField(max_length=100)

```

```
class Meta:
    constraints = [
        models.UniqueConstraint(
            Lower("name"),
            name="uq_department_name_ci",
        )
    ]
```

대소문자를 무시한 고유성을 표현할 수 있습니다.

그 밖에 주요 인수는 다음과 같습니다.

- `fields`: 고유성 조합에 포함할 필드
- `condition`: 일부 행에만 적용할 조건
- `include`: covering index 에 추가할 비키 열
- `opclasses`: PostgreSQL 등의 연산자 클래스
- `nulls_distinct`: 여러 NULL 을 서로 다른 값으로 볼지 설정
- `deferrable`: 트랜잭션 종료 시점까지 검사 연기
- `violation_error_code`: 모델 검증 오류 코드
- `violation_error_message`: 모델 검증 오류 메시지

데이터베이스에 따라 일부 옵션을 지원하지 않을 수 있습니다.

5. CheckConstraint

행이 특정 논리식을 만족하도록 제한합니다.

```
from django.db.models import Q

class CourseSection(models.Model):
    capacity = models.IntegerField()
    enrolled_count = models.IntegerField(default=0)

    class Meta:
        constraints = [
```

```

models.CheckConstraint(
    condition=Q(capacity__gte=1),
    name="ck_section_capacity_positive",
),
models.CheckConstraint(
    condition=Q(enrolled_count__gte=0),
    name="ck_section_enrolled_nonnegative",
),
models.CheckConstraint(
    condition=Q(enrolled_count__lte=models.F("capacity")),
    name="ck_section_enrolled_within_capacity",
),
]

```

NULL 을 허용하는 열은 NULL 의 의미까지 조건에 반영해야 합니다.

```

models.CheckConstraint(
    condition=Q(age__isnull=True) | Q(age__gte=18),
    name="ck_student_age_null_or_adult",
)

```

제약조건의 이름은 데이터베이스 전체에서 충돌하지 않도록 명확하게 짓는 것이 좋습니다. Django 제약조건의 세부 기능은 [공식 Constraints 레퍼런스](#)에 정리되어 있습니다.

6. violation_error_code 와 violation_error_message

```

models.CheckConstraint(
    condition=Q(credits__range=(1, 6)),
    name="ck_course_credits_range",
    violation_error_code="invalid_credits",
    violation_error_message="학점은 1 에서 6 사이여야 합니다.",
)

```

full_clean() 등 모델 검증 과정에서 더 명확한 오류를 제공하는 데 사용합니다. 데이터베이스가 직접 위반을 발견하면 일반적으로 IntegrityError 가 발생합니다.

7. unique_for_date, unique_for_month, unique_for_year

```
title = models.CharField(
    max_length=200,
    unique_for_date="published_at",
)
published_at = models.DateTimeField()
```

같은 날짜에 같은 제목을 허용하지 않는 식의 검증입니다.

다만 이것들은 validate_unique()가 검사하는 Django 수준 규칙이며 데이터베이스 제약조건이 아닙니다. 동시 요청을 포함해 강하게 보장해야 하는 규칙이라면 데이터베이스 UniqueConstraint 설계를 우선 검토해야 합니다.

8. validator, clean(), 제약조건의 차이

무결성 검사는 여러 계층에서 수행할 수 있습니다.

계층	예	장점	한계
폼	ModelForm.clean()	사용자 친화적 오류	폼을 거치지 않으면 우회
필드 validator	MinValueValidator	필드 단위 재사용	자동 save() 검증 아님
모델	Model.clean()	여러 필드 간 규칙 표현	DB 직접 입력·bulk 연산에서 우회
DB 제약조건	CheckConstraint	모든 입력 경로에 강제	복잡한 업무 규칙 표현에 제한
서비스 계층	등록 서비스 함수	여러 객체·외부 시스템 조정	모든 코드가 서비스를 사용해야 함

예를 들어 종료일이 시작일보다 늦어야 한다면 모델 검증을 작성할 수 있습니다.

```
from django.core.exceptions import ValidationError

class Semester(models.Model):
    starts_on = models.DateField()
    ends_on = models.DateField()
```

```
def clean(self):
    super().clean()

    if self.ends_on < self.starts_on:
        raise ValidationError({
            "ends_on": "종료일은 시작일보다 빠를 수 없습니다."
        })
```

그리고 가능한 경우 데이터베이스 제약도 추가합니다.

```
class Meta:
    constraints = [
        models.CheckConstraint(
            condition=models.Q(ends_on__gte=models.F("starts_on")),
            name="ck_semester_valid_period",
        )
    ]
```

중요한 원칙은 다음과 같습니다.

사용자에게 좋은 오류를 보여주는 검증과, 잘못된 데이터의 저장 자체를 막는 데이터베이스 제약을 함께 사용한다.

7. 동시성과 트랜잭션

1. 동시성과 트랜잭션

검사 후 저장하는 방식만으로는 동시 요청에 안전하지 않을 수 있습니다.

```
if not Enrollment.objects.filter(
    student=student,
    section=section,
).exists():
    Enrollment.objects.create(
        student=student,
        section=section,
    )
```

두 요청이 동시에 exists()를 통과하면 중복 행이 생길 수 있습니다. 따라서 고유 제약조건을 두고 예외를 처리해야 합니다.

```
from django.db import IntegrityError, transaction

try:
    with transaction.atomic():
        Enrollment.objects.create(
            student=student,
            section=section,
        )
except IntegrityError:
    # 이미 등록되었거나 다른 DB 제약을 위반함
    ...
```

재고·잔액처럼 읽고 수정하는 연산에는 행 잠금이 필요할 수 있습니다.

```
from django.db import transaction
```

```

with transaction.atomic():
    section = (
        CourseSection.objects
        .select_for_update()
        .get(pk=section_id)
    )

    if section.enrolled_count >= section.capacity:
        raise ValueError("정원이 마감되었습니다.")

    section.enrolled_count += 1
    section.save(update_fields=["enrolled_count"])

```

단, IntegrityError 는 여러 제약 위반에서 발생할 수 있으므로 단순히 모든 오류를 "중복"으로 처리하지 말고 원인을 구분해야 합니다.

2. 인덱스 설계

제약조건과 인덱스는 관련 있지만 목적이 다릅니다.

- 제약조건: 올바른 데이터만 저장되도록 보장
- 인덱스: 주로 조회 성능 향상
- 고유 인덱스: 성능과 고유성 보장을 함께 수행

```

class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    section = models.ForeignKey(CourseSection, on_delete=models.CASCADE)
    enrolled_at = models.DateTimeField(auto_now_add=True)
    status = models.CharField(max_length=20)

    class Meta:
        indexes = [
            models.Index(
                fields=["section", "status"],
                name="idx_enroll_section_status",
            )
        ]

```

```

    ),
    models.Index(
        fields=["-enrolled_at"],
        name="idx_enroll_recent",
    ),
]

```

복합 인덱스의 필드 순서는 중요합니다. 예를 들어 (section, status) 인덱스는 일반적으로 다음 질의에 유용합니다.

```

Enrollment.objects.filter(
    section=section,
    status="ACTIVE",
)

```

하지만 status 만으로 검색하는 질의에서는 충분히 활용되지 않을 수 있습니다. 실제 운영 질의와 데이터 분포를 보고 EXPLAIN 으로 검증해야 합니다. Django 의 인덱스 옵션은 [공식 Index 레퍼런스](#)에서 확인할 수 있습니다.

3. 완성된 대학 수강 모델 예시

```

from django.conf import settings
from django.core.validators import MaxValueValidator, MinValueValidator
from django.db import models
from django.db.models import F, Q

class TimeStampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True

```



```

class Department(TimeStampedModel):
    code = models.CharField(max_length=10, unique=True)
    name = models.CharField(max_length=100, unique=True)

    class Meta:
        ordering = ["name"]
        indexes = [
            models.Index(fields=["name"], name="idx_department_name"),
        ]

    def __str__(self):
        return f"{self.code} {self.name}"

class Student(TimeStampedModel):
    class Status(models.TextChoices):
        ACTIVE = "ACTIVE", "재학"
        LEAVE = "LEAVE", "휴학"
        GRADUATED = "GRADUATED", "졸업"

    student_number = models.CharField(max_length=20, unique=True)
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    department = models.ForeignKey(
        Department,
        on_delete=models.PROTECT,
        related_name="students",
    )
    status = models.CharField(
        max_length=10,
        choices=Status,
        default=Status.ACTIVE,

```

```
)

def __str__(self):
    return f"{self.student_number} {self.name}"
```

```
class Professor(TimeStampedModel):
    employee_number = models.CharField(max_length=20, unique=True)
    name = models.CharField(max_length=100)
    department = models.ForeignKey(
        Department,
        on_delete=models.PROTECT,
        related_name="professors",
    )
```

```
class Course(TimeStampedModel):
    code = models.CharField(max_length=20, unique=True)
    title = models.CharField(max_length=200)
    credits = models.PositiveSmallIntegerField(
        validators=[
            MinValueValidator(1),
            MaxValueValidator(6),
        ]
    )
    department = models.ForeignKey(
        Department,
        on_delete=models.PROTECT,
        related_name="courses",
    )
```

```
class Meta:
    constraints = [
```

```
        models.CheckConstraint(
            condition=Q(credits__range=(1, 6)),
            name="ck_course_credits_1_6",
        )
    ]
```

```
class CourseSection(TimeStampedModel):
```

```
    course = models.ForeignKey(
        Course,
        on_delete=models.PROTECT,
        related_name="sections",
    )
    professor = models.ForeignKey(
        Professor,
        on_delete=models.PROTECT,
        related_name="sections",
    )
    year = models.PositiveSmallIntegerField()
    semester = models.PositiveSmallIntegerField()
    section_number = models.PositiveSmallIntegerField()
    capacity = models.PositiveIntegerField()
    enrolled_count = models.PositiveIntegerField(default=0)

    students = models.ManyToManyField(
        Student,
        through="Enrollment",
        related_name="sections",
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
```

```

        fields=[
            "course",
            "year",
            "semester",
            "section_number",
        ],
        name="uq_course_section_identity",
    ),
    models.CheckConstraint(
        condition=Q(semester__in=[1, 2]),
        name="ck_section_semester",
    ),
    models.CheckConstraint(
        condition=Q(capacity__gte=1),
        name="ck_section_capacity_positive",
    ),
    models.CheckConstraint(
        condition=Q(enrolled_count__lte=F("capacity")),
        name="ck_section_count_within_capacity",
    ),
]
indexes = [
    models.Index(
        fields=["year", "semester", "course"],
        name="idx_section_term_course",
    )
]

```

```
class Enrollment(TimeStampedModel):
```

```
    class Status(models.TextChoices):
```

```
        ACTIVE = "ACTIVE", "수강 중"
```

```
        CANCELLED = "CANCELLED", "취소"
```

```
COMPLETED = "COMPLETED", "이수"
```

```
student = models.ForeignKey(
    Student,
    on_delete=models.CASCADE,
    related_name="enrollments",
)
```

```
section = models.ForeignKey(
    CourseSection,
    on_delete=models.CASCADE,
    related_name="enrollments",
)
```

```
status = models.CharField(
    max_length=10,
    choices=Status,
    default=Status.ACTIVE,
)
```

```
score = models.DecimalField(
    max_digits=5,
    decimal_places=2,
    null=True,
    blank=True,
)
```

```
class Meta:
```

```
    constraints = [
        models.UniqueConstraint(
            fields=["student", "section"],
            name="uq_enrollment_student_section",
        ),
        models.CheckConstraint(
            condition=Q(score__isnull=True) | Q(score__range=(0, 100)),
            name="ckenrollment_score_0_100",
```

```

    ),
]
indexes = [
    models.Index(
        fields=["student", "status"],
        name="idx_enroll_student_status",
    )
]

```

이 모델에서 적용한 설계 원칙은 다음과 같습니다.

- 학과·학생·교수·교과목·강좌를 별도 엔터티로 분리
- 수강 신청을 명시적인 중간 모델로 표현
- 학번, 교직원 번호, 교과목 코드에 단일 고유 제약 적용
- 강좌 식별에는 복합 UniqueConstraint 적용
- 학점, 학기, 정원, 점수 범위에 CheckConstraint 적용
- 삭제 정책을 업무 의미에 따라 PROTECT 와 CASCADE 로 구분
- 자주 사용할 조회 조합에 복합 인덱스 적용
- validator 와 DB 제약조건을 함께 적용

4. 모델 설계 시 자주 하는 실수

1. null=True 와 blank=True 를 같은 의미로 생각하는 것
전자는 DB 저장, 후자는 Django 검증에 관한 옵션입니다.
2. 금액을 FloatField 로 저장하는 것
정확성이 필요한 금액은 DecimalField 가 적합합니다.
3. 변화하는 업무 데이터를 choices 에 고정하는 것
관리자가 추가·수정해야 한다면 별도 모델이 낫습니다.
4. 모든 데이터를 JSONField 하나에 저장하는 것
관계, 타입, 제약, 인덱싱이 약해집니다.
5. save()가 자동으로 모든 검증을 한다고 생각하는 것
save()는 full_clean()을 자동 호출하지 않습니다.
6. validator 만으로 무결성이 보장된다고 생각하는 것
SQL, bulk 연산, 잘못된 코드 경로에서 우회될 수 있습니다.

7. `on_delete=models.CASCADE` 를 습관적으로 사용하는 것
중요한 이력 데이터가 연쇄 삭제될 수 있습니다.
8. 가변 객체를 기본값으로 직접 지정하는 것
`default={}`가 아니라 `default=dict` 를 사용해야 합니다.
9. `ForeignKey` 객체의 ID 만 필요해도 관련 객체를 조회하는 것
`obj.department_id` 를 사용하면 추가 쿼리를 피할 수 있습니다.
10. 인덱스를 지나치게 많이 만드는 것
조회는 빨라질 수 있지만 쓰기와 저장 공간 비용이 증가합니다.
11. 애플리케이션 검사 후 저장만으로 중복을 막는 것
동시성 상황에서는 반드시 DB 고유 제약조건이 필요합니다.
12. 마이그레이션 파일을 배포하지 않는 것
모델 코드와 마이그레이션은 함께 형상 관리해야 합니다.

5. 핵심 정리

Django 모델 설계는 단순히 Python 클래스에 필드를 나열하는 작업이 아닙니다.
좋은 모델은 다음 네 층을 함께 고려합니다.

1. **의미적 설계**: 업무 개념과 관계를 정확히 분리한다.
2. **자료형 설계**: 값의 의미에 맞는 Field 를 선택한다.
3. **무결성 설계**: 기본 키, 외래 키, 고유 및 검사 제약조건을 설정한다.
4. **실행 설계**: ORM 쿼리, 인덱스, 트랜잭션, 동시성을 고려한다.

특히 중요한 원칙은 다음과 같습니다.

Django 의 `validator` 와 `clean()`은 친절한 검증을 담당하고, 데이터베이스의 `UNIQUE`, `CHECK`, `FOREIGN KEY` 제약은 최종 무결성을 담당하도록 설계하는 것이 바람직합니다.